

Подготовка к экзамену по C++ (ИТМО)

Тема: Метапрограммирование (constexpr, SFINAE, Concepts)

Hardcore Theory Edition

21 июня 2026 г.

1 constexpr: Математика до запуска

Ключевое слово `constexpr` (появилось в C++11) означает *"Это значение (или функцию) можно вычислить на этапе компиляции"*. Если вы напишете `constexpr int x = 5 * 10;`, компилятор сам посчитает 50 и просто впишет число 50 в итоговый .exe файл. Процессор вообще не будет тратить время на умножение!

Функции constexpr: Если функция помечена как `constexpr`, она может выполняться компилятором. Но есть условие: все аргументы, переданные в нее, должны быть известны на этапе компиляции.

```
1 constexpr int Square(int x) { return x * x; }
2
3 // Компилятор считает это сам. В ассемблере будет просто "int a = 100;"
4 constexpr int a = Square(10);
5
6 int b; std::cin >> b;
7 // А это будет считаться в рантайме (как обычная функция), т.к. 'b' заранее неизвестно!
8 int c = Square(b);
```

2 Метафункции (<type_traits>)

Обычная функция принимает переменные и возвращает значение. **Метафункция** принимает ТИПЫ (через шаблоны) и возвращает константы или другие типы! Они живут в библиотеке `<type_traits>`.

Например, метафункция `std::is_integral<T>`:

- Если передать `int`, её поле `::value` будет равно `true`.
- Если передать `double`, её поле `::value` будет равно `false`.

В современном C++ вместо долгого `std::is_integral<T>::value` пишут коротко: `std::is_integr`

3 SFINAE (Substitution Failure Is Not An Error)

Перевод: Ошибка подстановки не является ошибкой. Это главный и самый уродливый костыль метапрограммирования C++, который держался до 2020 года.

Проблема: У нас есть две шаблонные функции с одинаковым именем. Одна для целых чисел, другая для дробных. Как компилятору выбрать нужную? Если компилятор подставляет ваш тип `T` в шаблон, и получается синтаксический бред (например, попытка взять `::type` у того, у кого его нет), компилятор **НЕ ВЫДАЕТ ОШИБКУ**. Он просто говорит: *"Этот кандидат мне не подходит, вычеркиваю его из списка и ищу другую перегрузку"*.

Для искусственного создания таких "вычеркиваний" используется `std::enable_if_t<УСЛОВИЕ, ТИП>`. Если `УСЛОВИЕ = true`, она возвращает `ТИП`. Если `УСЛОВИЕ = false`, она не возвращает ничего (ошибка подстановки!), и компилятор молча игнорирует эту функцию.

Суть на пальцах (Жизненная аналогия)

SFINAE — это кастинг актеров. Режиссер (Компилятор) ищет актера на роль (вызов функции). Приходит актер (Тип `double`). Режиссер дает ему текст для роли целого числа (шаблон `enable_if_t<is_integral>`). Актер не может это прочитать (Ошибка подстановки). Режиссер **НЕ** убивает актера (Not An Error). Он просто говорит: "Следующий!" — и дает ему текст для роли дробного числа. Тот читает идеально. Кастинг пройден.

4 Concepts (C++20): Смерть SFINAE

Писать `enable_if_t` физически больно. Поэтому в C++20 добавили **Концепты**. Это то же самое SFINAE, но встроенное в язык человеческим синтаксисом через ключевое слово `requires`.

```
1 template <typename T>
2 requires std::is_integral_v<T>
3 void print(T x) { std::cout << "Integer!"; }
```

Если вы передадите `double`, компилятор просто скажет: *"Constraints not satisfied (Требования не выполнены)"*, и выберет другую перегрузку. Чисто, красиво, легко читается.